



Transformation & Action

Part 2

www.3aayaam.in

Disclaimer

- All Literature.
- No Concepts.
- Mostly Syntax.
- Important to solve Business Problems.

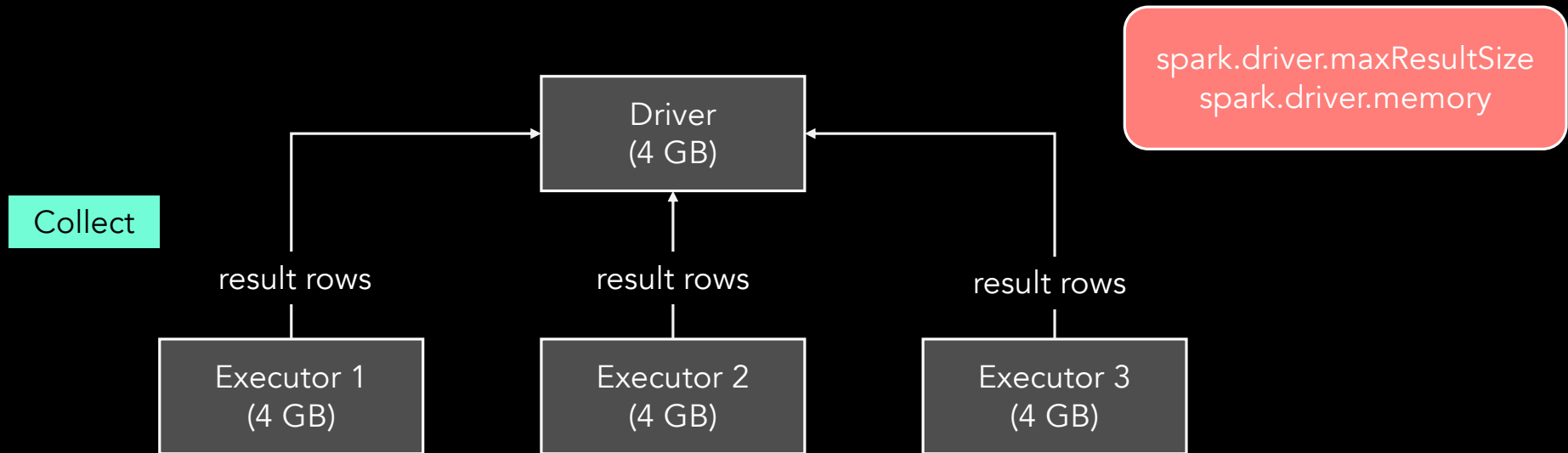
www.3aayaam.in

Agenda

1. `collect()`
2. `foreach()`, `foreachPartition()`
3. `replace()`
4. `when`, `otherwise`
5. `df.na.fill()`
6. Dataframe Iterator
7. Variable in PySpark

collect()

- All DataFrame records are collected as a Python List.
- collect() returns records from all executors to the driver.
- Use with smaller result set. Bigger result set can cause out of memory error.



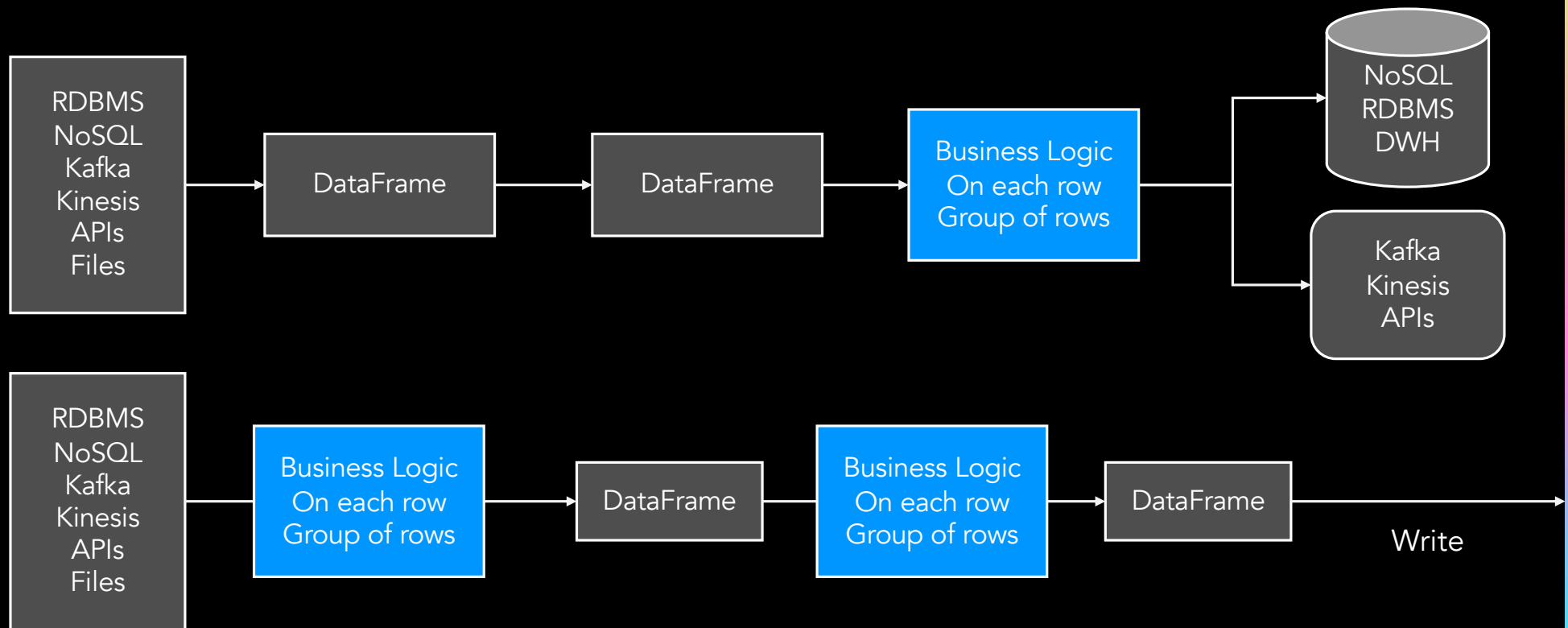
foreach, foreachPartition

- foreach – Apply custom defined Python function to each row of a DataFrame.
- foreachPartition - Apply custom Python function to an entire partition.
- To use logic that cannot be solved using DataFrame transformations.
- Example – Insert each row to a database, kafka etc., use aggregation logic per partition that cannot be done using Spark transformations.

```
def my_func(row):  
    some logic  
    some logic  
  
df1 = spark.read...load()  
df1.foreach(my_func)  
df1.foreachPartition(my_func)
```

- These APIs do not return data to driver.
- foreachPartition is more effective.

Usage



replace, expr, when & otherwise

- `replace()` – Can be used to replace one or multiple column values in a DataFrame.
- `df.replace( ["curr_val1", "curr_val2"], ["new_val1", "new_val2"], "<col-name>" )`
- `expr` is a function. Have to be imported from `pyspark.sql.functions`.
 - Similar to `selectExpr()`.
 - Can be used anywhere.
- `when` and `otherwise` are functions, not transformations. Have to be imported from `pyspark.sql.functions`.
 - Similar to SQL CASE function.
 - CASE has to be used with `selectExpr()` or `expr()`, `when+otherwise` can be used without these.

df.na.fill()

- Replace NULL values for one or more columns.
- `df.na.fill({"location": "Bangalore", "price": "0.00"})`

Dataframe Iterator

- Loop through all the records in a DataFrame.
- `Df.toLocalIterator()`
- Apply custom logic on the output result set rows.
- Memory footprint:
 - Records are sent to driver, just like `collect()`.
 - `collect()` creates a single list of all rows. `toLocalIterator()` loops through all rows in the result set and sends them one-by-one or partition-by-partition to driver.
 - Memory usage is size of largest partition in the dataframe.
 - Make sure to set `spark.driver.memory` as per requirement.

Iterator vs collect

Collect()

- Data from all partitions are retrieved as a single List data type.
- Driver receives all data from all executors.
- The entire result set must fit in Driver's memory.
- ALWAYS use with smaller result set/dataset.

toLocalIterator()

- Data from all partitions are retrieved as an Iterator data type.
- Data is sent one-by-one from individual partitions at a time.
- Only one partition's data must fit in Driver's memory.
- Alternative to Collect when large dataset has to be processed.

Use variables in PySpark

- Just like python variables.



Spark Functions

www.3aayaam.in

Disclaimer

- All Literature.
- No Concepts.
- Mostly Syntax.
- Important to solve Business Problems.

www.3aayaam.in

lit, expr, concat

- df.rdd – toLocalIterator, foreachPartition, foreach, df.rdd.getNumPartitions
- from pyspark.sql.functions import ...
- lit() – Used to pass constant value.
 - df.select('name', 'id', "graduated") – How "graduated" will be treated by Spark.
 - df.select('name', 'id', lit("graduated")) – "graduated" will be treated as Python constant by Spark.
 - Can be used anywhere.
- expr() – Function to execute SQL like expressions.
 - CASE WHEN
 - Mathematical functions
 - Concatenation
- concat() – Concatenate multiple column values from the same dataframe. concat(df.col1, df.col2, df.col3).

Date Functions

- `extract()` – Extract part of date or timestamp. "year", "month", "hour" etc.
 - `year()`, `month()`, `dayofmonth()`, `hour()`, `minute()`, `second()`, `weekofyear()`, `dayofyear()`, `dayofweek()`
- `date_add(<date>, <days>)` – Add days to date. `dateadd(<date>, <days>)`.
- `date_sub(<date>, <days>)` – Subtract days from date. `datediff(<date>, <days>)`.
- `date_diff(<date1>, <date2>)` – Difference in two days.
- `date_format(<date>, <format>)` – convert date or timestamp or date like string to the required format. Format = dd/MM/yy | yyyy-MM-dd | MM/dd/yyyy.HH.mm.ss
- `current_date()`, `current_timestamp()`, `now()`
- `to_date(<col>, <input_format_of_col>)`, `to_timestamp(<col>, <input_format_of_col>)`
- `unix_timestamp` – Convert any date/timestamp data to seconds
- `months_between`

When, Otherwise, Lower, Upper

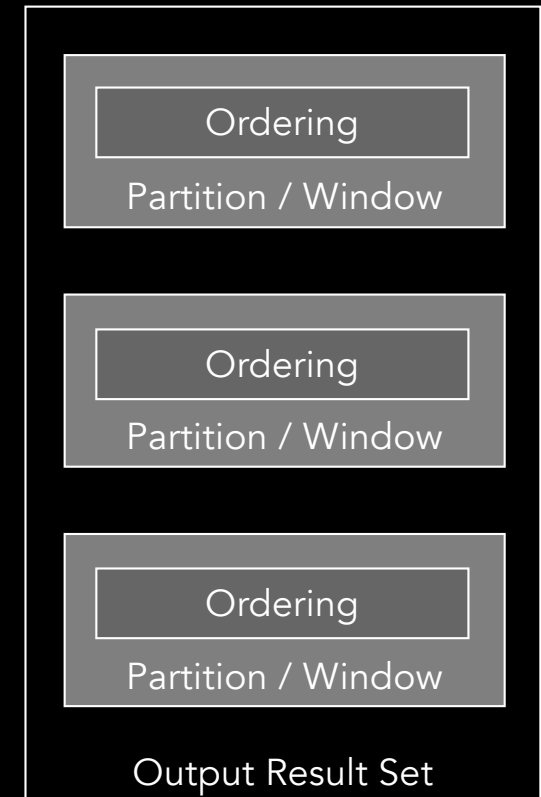
- Used to implement conditional logic.
- Similar to SQL CASE WHEN.
- `from pyspark.sql.functions import ...`
- `when(<condition>, <value1>).otherwise(<value2>)`
- `when(<condition1>, <value1>). when(<condition2>, <value2>).otherwise(<value3>)`
- `lower(<col>), upper(<col>)`
- Exercise:
 - if taxi duration is more than 180 mins, indicate "Airport Challenge".
 - If taxi duration is more than 60 and less than 180 mins, indicate "Operation Challenge".
 - If taxi duration is less than 60 then indicate "Airplane challenge".

Window Functions (from SQL)

- Partition or Window of result set
- PARTITION BY clause
- ORDER BY clause

```
agg_func(col) OVER (PARTITION BY col1, col2 ORDER BY col3, col4)
```

- Agg_func
 - avg, count, max, min, sum.
 - rank, row_number, dense_rank, percent_rank, first_value, last_value.



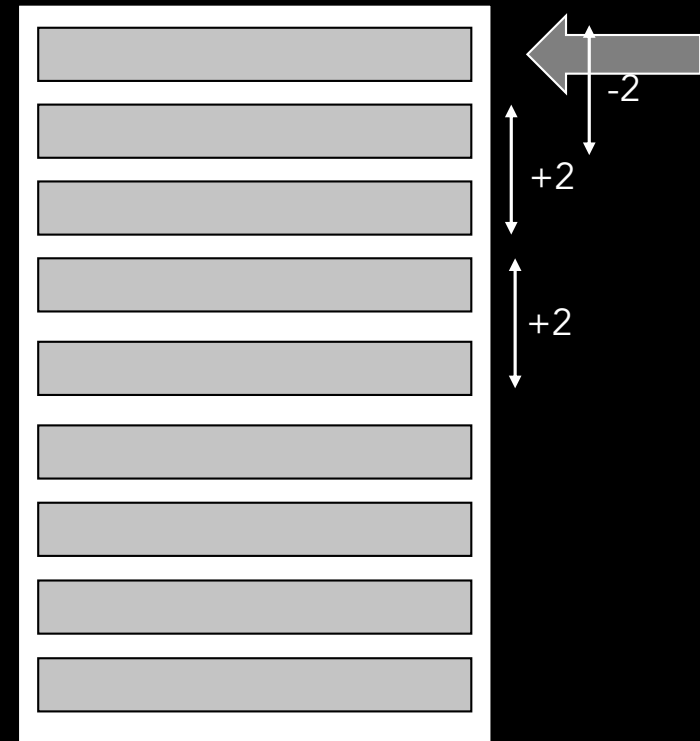
Window Functions

- Window has to be imported from pyspark.sql. `from pyspark.sql import Window`.
- `df.withColumn("<column-name>", <window function>)`
- `agg_func(col) OVER (PARTITION BY col1, col2 ORDER BY col3, col4)`
- Aggregate functions: `dense_rank()`, `rank()`, `percent_rank()`, `row_number()`, `nth_value()`
- `dense_rank().over( ... )`

```
dense_rank().over(Window.partitionBy("col").orderBy("col").rowsBetween(Window.currentRow,2))  
Window.unboundedPreceding  
Window.unboundedFollowing
```

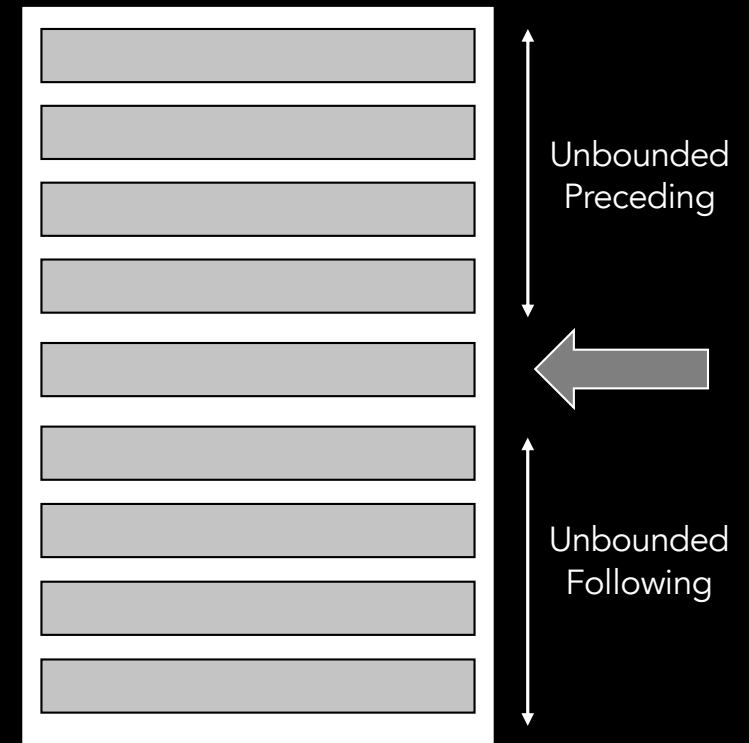
Window Functions

- (Window.currentRow, 2)
- (Window.currentRow, -2)



Window Functions

- (Window.currentRow, 2/-2)
- (Window.currentRow, Window.unboundedPreceding)
- (Window.currentRow, Window.unboundedFollowing)
- (unboundedPreceding, unboundedFollowing)



Window Functions - Exercise

- Identify the top 10 revenue-generating frequent fliers per country.
- Find the top 25 costly fairs for each airplane model for each passenger country. Which routes have the highest cost per aircraft model.
- Find the top 5 high speed aircrafts for each airlines.
- Find the longest route for each airplane model.
- Find out the most turbulent routes that each aircraft takes for individual airplane models.
- List the airports that make the flights wait most. Normal waiting time should not be more than 45 mins to 1 hour. But certain airports have more waiting time than others. Find the list of top 5 airports in each country.
- Find out the most used airplane model for the last week of any given day.
- Find the list of airplane models with least engine performance per route.
- Find out year-on-year revenue for each airline (airplane model). Revenue should be considered for the entire year.

Explode

- Will talk with JSON.

Summary

www.3aayaam.in

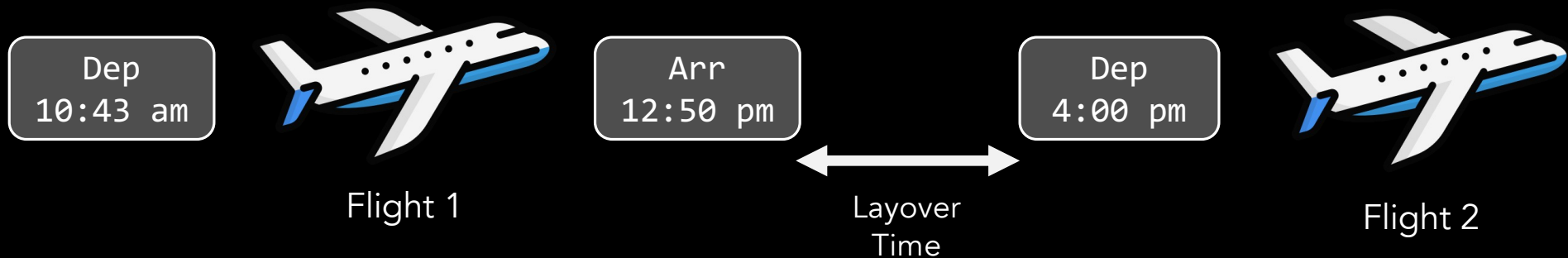
- collect, foreach, foreachPartition
- When & otherwise, df.na.fill
- Dataframe iterator, using Python variables
- lit, expr, concat
- Date functions
- Window functions
- Explode

Project 4

(Flight Transportation Data)

- Layover Analysis : Identify all possible connecting flights for individual passengers. For each connection, calculate the layover duration and flag connections that are too short (e.g., < 60 minutes) or excessively long (e.g., > 6 hours). **Calculate for only one layover.**
- Travel Activity : For each passenger, identify the time gap (in days) between their consecutive flights. This is to understand typical travel frequency, identify long periods of inactivity (potential churn indicators), or detect unusually rapid sequence of flights.
- Environmental Impact : As environmental regulations become stricter, there's increasing pressure to accurately measure and report carbon footprint. We need an automated, auditable system to precisely calculate CO2 emissions per flight, per aircraft, per route, and per passenger, adhering to international reporting standards (e.g., ICAO) to demonstrate our commitment to sustainability and avoid potential penalties.

Layover Analysis



- Considerations:
- Flights have to be consecutive and on the same day for layover. There can be duplicate rows in data. Duplicate rows have to be removed.
- Have to make sure that flight data is sorted on departure time.
- As one layover is relevant, arrival time of 1st flight and depart time of 2nd flight have to be compared. Difference of arrival time and depart time is needed for analysis.
- Flag the layover whose duration is less than 1hr and more than 6 hours.



Thank You
www.3aayaam.in